



CS-Simplified

Learning • Building • Sharing

C++ Programming Notes

Module 01

Introduction to C++

Beginner-Friendly Comprehensive Notes

Created by

Gaurav Kandpal

B.Tech Computer Science Engineering Student

C++ Programming Textbook Notes

Module 1: Introduction to C++, Programming Paradigms & OOP Concepts

Author: Gaurav Kandpal

Platform: CS-Simplified

Level: Beginner to Advanced

1. PROGRAMMING PARADIGMS

Conceptual Explanation

A **programming paradigm** defines the fundamental style, philosophy, and standard approach used to structure, organize, and execute instructions in a computer program. It provides developers with a mental model for translating complex real-world problems into software architecture.

Choosing the right programming language and understanding its underlying paradigm is essential because the paradigm dictates how state, memory allocation, logic execution, and data flow are governed.

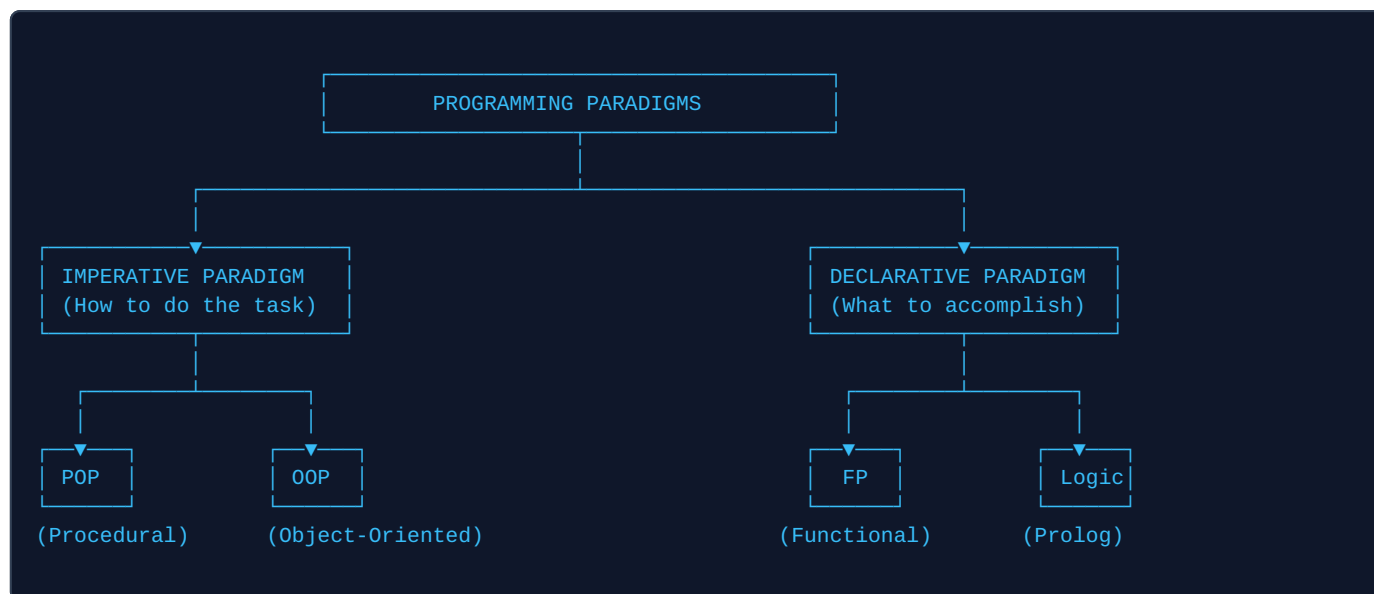
"Any fool can write code that a computer can understand. Good programmers write code that humans can understand."

— Martin Fowler

Why Paradigms Matter (Consequences of Unstructured Code):

- **Uncontrolled Complexity:** Code without structure quickly devolves into "spaghetti code," making execution pathways unpredictable.
- **Poor Readability & Maintainability:** Unstructured software becomes nearly impossible for development teams to audit, debug, or extend.
- **Fragility & High Bug Frequency:** Modifications in one component produce unintended side effects across unrelated modules.
- **Testing Hurdles:** Tightly coupled code prevents isolated unit testing and component validation.

Taxonomy of Programming Paradigms



- **Imperative Paradigm:** The developer explicitly dictates *how* the computer must achieve a result by executing sequential statements that directly mutate memory and program state.
 - **Procedure-Oriented Programming (POP):** Focuses on functions, routines, and algorithmic steps.
 - **Object-Oriented Programming (OOP):** Focuses on encapsulating data and logic into distinct runtime entities (objects).
- **Declarative Paradigm:** Focuses on *what* the computation should achieve without specifying step-by-step control flow (e.g., SQL queries, Functional Programming, Logic Programming).

2. PROCEDURE-ORIENTED PROGRAMMING (POP) VS. OBJECT-ORIENTED PROGRAMMING (OOP)

Procedure-Oriented Programming (POP)

In POP, the entire program is decomposed into smaller units called **functions** or **subroutines**. The primary focus is on algorithmic execution flow and procedural logic.

Memory & Data Access Mechanics in POP: POP typically relies on global variables so multiple functions can process shared data. Because POP lacks access control specifiers (such as `public` or `private`), any function in the codebase can access and alter global memory. This lack of data hiding makes procedural programs vulnerable to unauthorized modifications and difficult to debug in large software systems.

POP Design Flow: Follows a **Top-Down Approach**, starting with main program logic and breaking it down into sub-functions.

Object-Oriented Programming (OOP)

In OOP, the program is constructed around **objects**—fundamental entities representing instances of user-defined data structures called **classes**. The focus shifts from function sequence to data protection and domain modeling.

Memory & Security Mechanics in OOP: Data is bound directly to member functions inside a class container. Access specifiers regulate access to data members, preventing outside functions from corrupting internal object state.

OOP Design Flow: Follows a **Bottom-Up Approach**, defining foundational classes/objects first and composing them into larger application systems.

POP vs. OOP Feature Matrix

Feature	Procedure-Oriented Programming (POP)	Object-Oriented Programming (OOP)
Core Focus	Functions, algorithms, and step-by-step procedural logic.	Data protection, objects, and modular structure.
System Architecture	Top-Down Approach (Main program divided into routines).	Bottom-Up Approach (Building blocks assembled into systems).
Data Security	Low. Global data is vulnerable to unintended modifications.	High. Data abstraction and encapsulation hide sensitive state.
Access Specifiers	None. Functions have open access to global variables.	Supported (private, public, protected).
Code Reuse	Low. Relies on function calls; difficult to extend safely.	High. Extensible via Inheritance and Polymorphism .
Language Examples	C, FORTRAN, Pascal, Basic.	C++, Java, C#, Python, Simula.

3. CORE PRINCIPLES & CONCEPTS OF OOP

Object-Oriented Programming is grounded in six fundamental principles. Understanding how these concepts operate in memory and C++ code is crucial for writing professional software.

1. Class (The Blueprint)

A **Class** is a user-defined blueprint or data type definition containing data members (variables) and member functions (methods).

Memory Footprint: Defining a class allocates *0 bytes* of RAM at runtime. It serves as a schema instructing the compiler how to lay out memory when an object is created.

2. Object (The Instance)

An **Object** is a runtime instance of a class. When an object is instantiated, physical RAM is allocated to hold its specific data attributes.

```
[ RAM Memory Layout of Class vs Object ]

Class Definition (Compiler Schema - 0 Bytes allocated in RAM):
class Student { int id; double gpa; };

Instantiation (Allocates 12 Bytes in RAM):
Student s1;

RAM Block (Address: 0x7ffd10):

┌────────── int id (4 Bytes) ───────────┐ ┌────────── double gpa (8 Bytes) ───────────┐
│                                         │ │                                         │
└──────────┬──────────┬──────────┬──────────┘ └──────────┬──────────┬──────────┬──────────┘
Address: 0x7ffd10                                         Address: 0x7ffd14
```

3. Encapsulation (Data Binding & Hiding)

Encapsulation is the wrapping of data members and functions into a single class container, combined with access restrictions. By marking data as `private`, the state cannot be altered directly from outside code, ensuring data integrity.

4. Abstraction (Complexity Hiding)

Abstraction refers to presenting only the essential interface to the user while concealing complex background details and internal execution mechanisms.

5. Inheritance (Code Reuse & Hierarchy)

Inheritance allows a new class (derived/child class) to inherit attributes and behaviors from an existing class (base/parent class), promoting code reuse and establishing class hierarchies.

6. Polymorphism (Multiple Forms)

Polymorphism (Greek for "many forms") allows a single interface, function, or operator to exhibit different behaviors depending on data types or dynamic context.

- **Compile-Time Polymorphism:** Resolved during compilation (Function Overloading, Operator Overloading).
- **Run-Time Polymorphism:** Resolved dynamically during execution (Function Overriding via Virtual Functions).

4. INTRODUCTION TO C++ & ARCHITECTURAL FEATURES

Origin & Historical Context

C++ was developed by Swedish computer scientist **Bjarne Stroustrup** at **AT&T Bell Laboratories** in Murray Hill, New Jersey, starting in 1979–1980.

Stroustrup was impressed by the software structure and abstractions of **Simula 67** (the first OOP language), but found Simula too slow for high-performance systems. Conversely, the **C language** was fast and gave direct access to hardware memory, but lacked abstraction tools for large applications.

To solve this, Stroustrup combined Simula 67's object-oriented paradigms with C's speed and direct memory manipulation. The initial result was called "**C with Classes**", which was formally renamed to **C++** in 1983.

Etymology of C++

The name comes from the C unary increment operator `++`, signifying that C++ is an "incremented" and enhanced evolution of C.

Core Features of C++

- **Multi-Paradigm Language:** C++ does not force a single style. Developers can write Procedural, Object-Oriented, Functional, or Generic (Template-based) code within the same project.
- **Middle-Level Language:** Combines high-level abstractions (classes, exception handling, STL) with low-level direct memory manipulation (pointers, explicit dynamic allocations).
- **Case-Sensitive:** Identifiers strictly differentiate case (e.g., `VALUE`, `Value`, and `value` are distinct).
- **Zero-Overhead Principle:** C++ features are designed so that what you don't use, you don't pay for in memory or execution time.

Comprehensive C++ OOP Code Example

```
// C++ Demonstration: Class, Encapsulation, and Object Allocation
#include <iostream>
#include <string>

class BankAccount {
private:
    double balance; // Encapsulated state (Private data member)

public:
    // Parameterized Constructor
    BankAccount(double initialBalance) {
        balance = (initialBalance >= 0) ? initialBalance : 0.0;
    }

    // Setter method with validation
    void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            std::cout << "Deposited: $" << amount << std::endl;
        }
    }

    // Const Getter method (Abstraction)
    double getBalance() const {
        return balance;
    }
};

int main() {
    // Instantiating object in Stack Memory
    BankAccount userAccount(1000.00);

    userAccount.deposit(500.25);
    std::cout << "Final Account Balance: $" << userAccount.getBalance() << std::endl;

    return 0;
}
```

5. COMPILER VS. INTERPRETER

Conceptual Explanation

CPUs understand only low-level binary machine instructions (0s and 1s). High-level C++ code must be translated into machine code before execution. Translation engines fall into two primary categories: **Compilers** and **Interpreters**.

Compiler (C, C++, Rust)

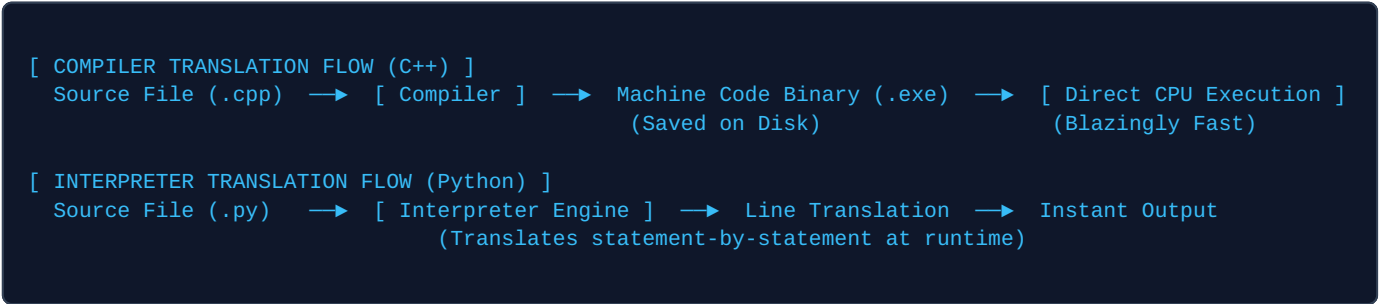
A **Compiler** scans and translates the complete high-level source program into a standalone native machine binary executable (e.g., .exe or Linux ELF binary) in a single pass before execution.

- **Execution Phase:** Compilation occurs completely prior to program run. Once compiled, the binary runs natively on hardware at maximum speed.
- **Error Checking:** Errors are caught during *compile-time*. If syntax or type errors exist, binary generation fails completely.

Interpreter (Python, JavaScript, Ruby)

- An **Interpreter** translates and executes source code dynamically, statement-by-statement or line-by-line during runtime.
- **Execution Phase:** Translation and execution happen concurrently. No standalone executable binary is generated on disk.
 - **Error Checking:** Errors are discovered at *runtime* when execution encounters the specific error statement.

Execution Flow Comparison Diagram



Compiler vs. Interpreter Summary Matrix

Metric	Compiler (e.g., C++)	Interpreter (e.g., Python)
Translation Mechanism	Translates entire source file at once.	Translates code line-by-line during execution.
Execution Speed	Fast performance (native binary runs on hardware).	Slower performance (runtime interpretation overhead).
Output File	Generates separate executable binary files.	No intermediate object or binary files created.
Error Detection	Compile-Time: All errors listed before run.	Runtime: Halts when error line is reached.
Target Use Cases	Game engines, operating systems, embedded systems.	Rapid scripting, data science, web backend logic.

Common Gotcha: C++ is Backward Compatible with C

Because C++ is a superset of C, almost all valid C code will compile under a C++ compiler. However, writing idiomatic modern C++ requires adopting classes, references, standard containers, and smart pointers rather than raw C-style pointers and `malloc()`.

Best Practice: Compiler Diagnostics

Always enable compiler warnings during development (e.g., `g++ -Wall -Wextra -std=c++20 main.cpp`). Modern C++ compilers act as severe static analysis tools that catch bugs before runtime!

❤️ Thank You

Thank you for taking the time to read these notes.
I hope they made learning simpler, clearer,
and more enjoyable.



Created by
Gaurav Kandpal



CS-Simplified
Learning • Building • Sharing

Connect with Me



github.com/gauravkandpalofficial



linkedin.com/in/gaurav-kandpalofficial

★ If these notes helped you, consider giving the repository a star and sharing it with others. Your support motivates me to continue creating beginner-friendly Computer Science notes and projects.